

Pontifícia Universidade Católica do Paraná

Integrantes Projeto RA2 Equipe 2:
Adriano Proença Barbosa
Bernardo Fadel Braga de Sampaio
Eduardo Fernandes Lopes Veiga
João Augusto Santin Kraft
Matheus Alberti da Silva
Professor (a): Andreza de Sousa Vieira

Disciplina: Design de Software
PUCaronas - Sistema de Caronas Universitárias

Curitiba
2026

1. Identificação e contextualização

1.1 Domínio da Aplicação

O PUCaronas é um sistema de caronas universitárias voltado à estudantes exclusivamente da PUCPR. A aplicação permite organizar a oferta de caronas e a solicitação de vagas por passageiros.

1.2 Problema Abordado

Estudantes que fazem trajetos semelhantes muitas vezes se deslocam separadamente, aumentando os custos de transporte. O sistema procura facilitar o compartilhamento de caronas e evitar problemas como solicitações duplicadas, falta de vagas, alterações indevidas em uma solicitação e principalmente melhorar a segurança dos estudantes.

1.3 Público-Alvo

Motoristas: estudantes que oferecem vagas em seus veículos. Passageiros: estudantes que procuram caronas para a universidade.

1.4 Objetivo do Sistema

O objetivo é permitir o cadastro, a solicitação e o gerenciamento de caronas, incluindo valores, pagamentos e notificações. O trabalho dá continuidade ao projeto PUCaronasJavaFX, desenvolvido na disciplina de Programação Orientada a Objetos, com melhorias na organização do código e aplicação de padrões de projeto GoF.

2. Descrição da solução

O sistema utiliza Java e JavaFX, com organização em camadas para separar interface, regras de negócio e persistência. Os dados são armazenados localmente por serialização. Entre as funcionalidades estão o cadastro de usuários, motoristas, veículos e caronas, solicitação e aceite ou recusa de vagas, cancelamento, cálculo do preço, pagamento, histórico e notificações. As principais regras impedem solicitações duplicadas ou sem vagas, controlam as mudanças de estado das solicitações e evitam pagamentos repetidos. A classe CaronasFacade coordena essas operações.

3. Padrões de projeto utilizados

Foram utilizados seis padrões GoF. Cada um foi escolhido para resolver uma necessidade específica do projeto.

3.1 Strategy

Intenção e problema: permitir diferentes formas de calcular o preço de uma carona sem concentrar todas as fórmulas no mesmo método.

Justificativa e aplicação: as políticas de preço são implementadas separadamente e selecionadas conforme a modalidade da carona.

Classes: `PoliticaPreco`, `PrecoFixoStrategy`, `PrecoPorKmStrategy`, `PrecoRateioStrategy`, `PoliticaPrecoFactory` e `CaronasFacade`.

Evidência no código: `CaronasFacade.simularPreco()` utiliza

`PoliticaPrecoFactory.criar(...).calcular(...)`.

Benefício: facilita a manutenção e a inclusão de novas regras de preço.

3.2 State

Intenção e problema: controlar o comportamento de solicitações pendentes, aceitas, recusadas e canceladas, impedindo transições inválidas.

Justificativa e aplicação: cada situação possui uma classe de estado que define as transições permitidas.

Classes: `SolicitacaoCarona`, `SolicitacaoState`, `PendenteState`, `AceitaState`, `RecusadaState` e `CanceladaState`.

Evidência no código: `SolicitacaoCarona.aceitar()`, `recusar()` e `cancelar()` delegam as ações ao objeto retornado por `estado()`; `mudarEstado(...)` atualiza o estado.

Benefício: centraliza as regras de transição e reduz alterações indevidas.

3.3 Observer

Intenção e problema: informar acontecimentos das caronas sem misturar a criação de notificações às operações principais.

Justificativa e aplicação: observadores recebem eventos publicados no barramento.

Classes: `EventoObserver`, `NotificacaoObserver`, `EventoCarona`, `EventBus` e `CaronasFacade`.

Evidência no código: `EventBus.registrar(...)` adiciona observadores e

`EventBus.publicar(...)` chama `atualizar(...)` em cada observador.

Benefício: separa a comunicação de eventos das regras de caronas.

3.4 Singleton

Intenção e problema: garantir um único barramento compartilhado para os eventos da aplicação.

Justificativa e aplicação: o EventBus mantém uma instância estática e controla o acesso a ela.

Classe: EventBus. Evidência no código: declaração `private static final EventBus INSTANCE` e método público `EventBus.getInstance()`.

Benefício: evita barramentos independentes para as mesmas notificações.

3.5 Facade

Intenção e problema: simplificar o acesso às operações que envolvem serviços e repositórios diferentes.

Justificativa e aplicação: a fachada reúne as ações principais relacionadas às solicitações e aos pagamentos.

Classes: CaronasFacade e os serviços e repositórios coordenados por ela.

Evidência no código: `CaronasFacade.solicitar(...)`, `aceitar(...)`, `recusar(...)`, `cancelar(...)` e `pagar(...)`.

Benefício: reduz a necessidade de a interface conhecer os detalhes de cada operação.

3.6 Template Method (estrutura de telas)

Intenção e problema: reaproveitar etapas comuns da montagem das telas CRUD sem duplicar a estrutura em cada tela.

Justificativa e aplicação: a classe-base define a sequência de inicialização e as subclasses fornecem etapas específicas.

Classes: BaseCrudView e suas subclasses de telas CRUD.

Evidência no código: `BaseCrudView.inicializarTela()` organiza a inicialização, enquanto `inicializarController()` e `criarTela()` são métodos abstratos implementados pelas subclasses.

Benefício: diminui a repetição de código na interface JavaFX.

4. Análise de aplicabilidade dos padrões GoF

Além dos padrões utilizados, foram analisados os demais padrões GoF. A tabela apresenta os motivos pelos quais eles não foram aplicados nesta versão do PUCaronas.

Padrão	Justificativa
Abstract Factory	Não há famílias de objetos relacionadas que precisem ser criadas em conjunto.
Builder	A criação das entidades não exige montagem complexa por etapas.
Factory Method	A seleção das políticas utiliza uma fábrica simples, sem uma hierarquia de criadores.
Prototype	Não há necessidade de copiar objetos configurados.
Adapter	Não há integração externa com interfaces incompatíveis.
Bridge	O sistema não possui duas hierarquias que precisem variar separadamente.
Composite	Não há estrutura hierárquica em árvore que exija tratamento uniforme.
Decorator	Não é necessário acrescentar comportamentos dinamicamente aos objetos.
Flyweight	Não há grande volume de objetos imutáveis repetidos que justifique compartilhamento de estado intrínseco.
Proxy	Não há necessidade atual de acesso remoto, carregamento tardio ou controle por representante.
Chain of Responsibility	As validações são ligadas diretamente ao fluxo de caronas; uma cadeia acrescentaria complexidade.
Command	A interface não exige fila de comandos ou funções de desfazer e refazer.
Interpreter	O sistema não utiliza linguagem ou gramática própria.
Iterator	As coleções do Java já oferecem os recursos de iteração necessários.

Mediator	A CaronasFacade já coordena o fluxo; outro mediador repetiria essa função.
Memento	Não existe requisito de restaurar estados anteriores
Visitor	Não há necessidade de adicionar frequentemente operações externas sobre uma estrutura fixa de tipos.

5. Diagrama de classes

O diagrama de classes apresenta as principais classes e seus relacionamentos, destacando os padrões de projeto utilizados no sistema PUCaronas. Para visualizar o diagrama com melhor resolução e facilitar a leitura, segue o link do Google Drive abaixo:

Link: https://drive.google.com/file/d/1DqFroX1I5BUGa3S1Sl9f_VaBVWgR7Xhx/view?usp=sharing

6. Reflexão e lições aprendidas

A principal dificuldade foi transformar um projeto focado em CRUD em um sistema com regras de negócio mais organizadas. Para isso, usamos padrões como State para controlar os status, Strategy para as regras de preço, Observer e Singleton para as notificações e Facade para centralizar o fluxo do sistema. Também evitamos usar padrões sem necessidade, como Visitor, Memento, Flyweight e Bridge. Como melhorias futuras, o projeto pode evoluir na persistência dos dados, nos testes e na integração completa com a interface JavaFX.